

Understanding Offline Evaluations (Offline Evals)

Definition & Context

- **Offline Evaluation** represents any evaluation pipeline that you run on your LLM-based application **before deploying it to production**.
- There is nothing structurally new you need to learn to understand offline evaluations. Every evaluation setup and pipeline we built in my previous sessions (including the UPSC grading mock test application) is a direct example of an offline eval.
- In that UPSC application, we evaluated mock essay exams like a human grader by building a **golden dataset** and utilizing the **LLM as a Judge** method. Because that evaluation happened after the software was built but **prior to its deployment**, it was an offline evaluation.

The Three Core Benefits & Purposes of Offline Evals

1. Pre-Release Testing & Release Gates

- You cannot deploy an LLM-based software into production without testing it first. Doing so exposes your business to extreme operational risks. Real-world case studies like the Air Canada and ChatGPT failures highlight the dangers of deploying untested LLM systems.
- In modern LLM Operations (LLMOps), this process is automated. We can set up an automated **release gate** in our **CI/CD** pipelines via GitHub Actions.
- When developer code is pushed to GitHub, the CI pipeline triggers our offline evaluation script against our golden dataset to output an evaluation score. If the score is above a pre-defined threshold (e.g., **95%**), the deployment pipeline is automatically triggered to push the code live. If the score falls below **95%**, the deployment is blocked, a failure notification is sent, and we can roll back to the previous stable version of the software.

2. Version Comparison (Leveling the Playing Field)

- Offline evaluations allow you to make clean, objective design choices when multiple development possibilities exist.
- For instance, if you are choosing whether to build my UPSC grader using **Claude** or **OpenAI's models**, you can create two distinct versions of the application. You keep everything else—your system prompts, databases, and application code—exactly identical.
- By running both versions against the exact same static **golden dataset**, the playing field is entirely level. The resulting scores allow you to easily identify which model performs better for your specific task.
- This version-comparison approach is invaluable when choosing between different **system**

prompts, rerankers, vector databases, or entire application architectures.

3. Regression Testing (Tests of Change)

- **Regression testing** is a "test of change". When updating a live application (Version 1), you must verify that making a change to improve one specific area does not accidentally break or degrade performance in another area—this degradation is called a **regression**.
 - *Example:* Suppose students interacting with my CampusX chatbot are asking about refunds, and we observe in production that the chatbot answers in a cold tone. To improve student relations, I modify my system prompt, instructing the bot to be exceptionally kind, soft, and polite.
 - *The Regression:* If the bot's tone becomes too soft, it might compromise factual accuracy to please the user. When asked about the price of our Insider Plan (which is exactly 19,500), the bot might state "It's around 19,000" to make the price sound gentler.
 - To prevent this, we run offline evals on a diverse **golden dataset** containing a wide range of test cases (refund questions, pricing questions, and course curriculum questions). If our prompt change keeps the refund success rate high but causes the pricing success rate to drop from 90% to 80%, we have caught a regression and know the change should not be deployed. Any small change in system prompts, models, or vector databases must undergo this test to ensure nothing else is broken.
-

The Reality of Production: Why Offline Evals Are Not Enough

Once your LLM application is deployed to production, it faces three major operational risks that offline evaluations are completely blind to:

1. Unanticipated Inputs:

- In an offline environment, we test our system on a small, curated golden dataset of 200 to 500 questions we anticipate users will ask.
- In production, the system is exposed to real-world, uncontrolled users who will input queries you never anticipated or tested for. This includes:
 - **Language Mixing (e.g., Hinglish):** Users suddenly mixing Hindi and English, when the system was evaluated primarily on English.
 - **Ambiguous or Half-Questions:** Incomplete or highly vague queries.
 - **Angry Rants:** Long, emotionally charged complaints where the actual question is buried under frustration.
 - **Adversarial Prompt Injections:** Users actively trying to bypass the chatbot's system instructions to extract unauthorized or harmful outputs.
 - **Complex Edge Cases:** Highly specific scenarios that never appeared in the test

dataset.

2. Emergent and Systematic Failures:

- These are systemic failures that only manifest at scale in production and cannot be replicated offline.
- *Scale-Related Latency*: If I launch a new course and thousands of concurrent users flood my chatbot, the system's **latency** might spike dramatically. This cannot be tested offline.
- *Subtle Biases*: A systematic bias (e.g., the chatbot acting colder or less helpful toward students from non-technical backgrounds) is a pattern that only becomes visible after analyzing thousands of live production conversations.

3. Data Drift:

- Over time, your business and its environment evolve. At CampusX, we will periodically change course prices, adjust curricula, and update policies.
- If we keep using the original offline golden dataset and evaluation pipeline we created on day one, our offline evaluation scores will continue to look outstanding. However, because the underlying production documentation has changed (**drift**), our live users will experience incorrect chatbot responses and leave heavy negative feedback. The offline evaluation has become **obsolete** because the data distribution has shifted.

The Critical Offline Limitation

Offline evaluations require a static **ground truth/answer key** (contained in the golden dataset) to calculate correctness. In production, we do not have an answer key for live user queries because we cannot predict what users will ask, nor do we know the correct responses in advance. This is why **Online Evals** are mandatory.

Understanding Online Evaluations (Online Evals)

Definition & Core Philosophy

- **Online Evaluation** is the process of evaluating your system on **live production traffic after deployment** as real users actively interact with it.
- Its most critical characteristic is that **it operates without an answer key or a golden dataset**.
- Offline and online evaluations are not rivals; one does not replace the other. They are **complementary** and must run together.
- **The Core Distinction**:
 - **Offline Evals** verify if your application is working **correctly** (Correctness).
 - **Online Evals** verify if your application is running **normally** in production (Normalcy).

Illustrating Correctness vs. Normalcy (UPSC Grader Example)

- **Offline Correctness:** We evaluate correctness by directly comparing the grades our LLM grader assigns to answers against the grades assigned by a professional human evaluator. If they match closely, our system is correct.
- **Online Normalcy:** In production, we cannot measure correctness this way because the live student answers have never been graded by a human; we have no human marks to compare against. Instead, we monitor **normalcy**:
 - We record and store the total score of every paper our system grades in production.
 - We aggregate these scores and plot their **score distribution** over a weekly window.
 - If the distribution is highly stable across Week 1, Week 2, Week 3, and Week 4 (e.g., averaging around 500, with some at 700, and a few at 200), this becomes our **baseline**.
 - If in Week 5 the distribution suddenly spikes dramatically, with scores clustering around 800 and 900, we know the system is behaving abnormally compared to our baseline. This shift triggers an investigation to see if there is a grading logic failure.

Online Correctness Workarounds (User Feedback)

If we must track correctness online without an answer key, we use **user feedback** as an alternative signal. By adding a thumbs-up/thumbs-down feature to our chatbot, we can monitor user behavior. A sudden spike in thumbs-down ratings over the past hour tells us the chatbot is outputting incorrect answers, serving as a proxy for correctness.

Comparative Summary: Offline vs. Online Evals

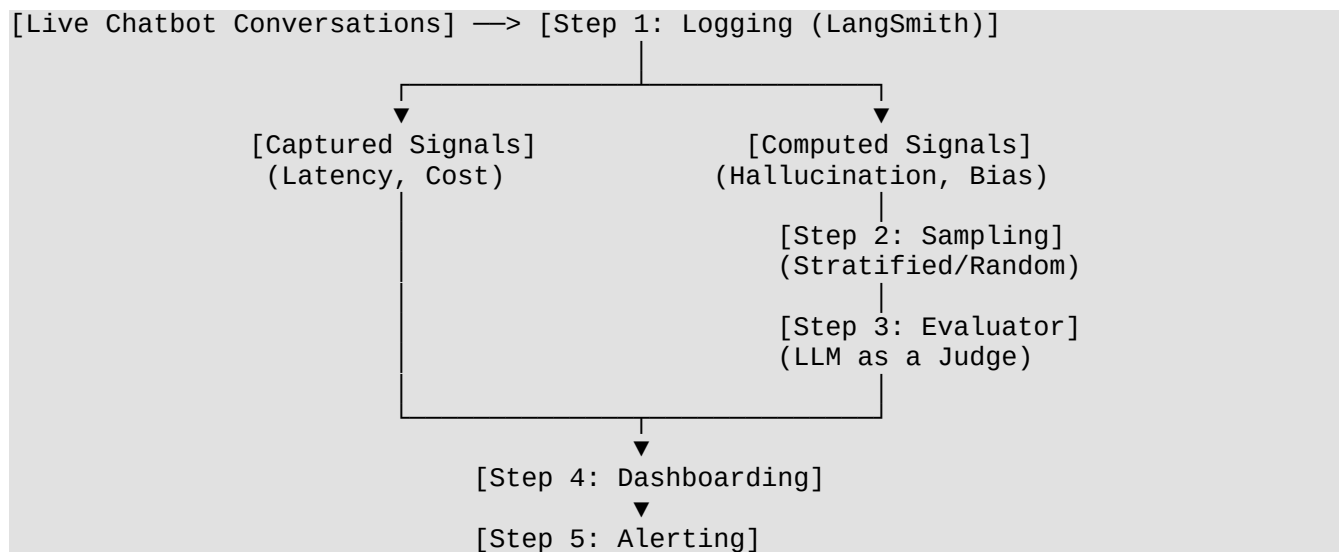
Comparative Summary: Offline vs. Online Evals

Feature / Dimension	Offline Evaluations	Online Evaluations
Timing	Executed before deployment	Executed after deployment and runs consistently
Data Source	Static, pre-defined Golden Dataset	Live, real-time production traffic
Answer Key	Relies on a pre-defined ground truth/answer key	Operates without an answer key (must estimate quality)
Input Type	Anticipated inputs curated by developers	Unanticipated, uncontrolled inputs from real users
Key Detections	Catches regressions (tests of change)	Catches drift, systematic/emergent bugs, and surprises
Primary Use Cases	CI/CD release gates and	Monitoring live system health

Feature / Dimension	Offline Evaluations	Online Evaluations
	version comparisons	and drift detection
Cost & Speed	Fast, cheap, and repeatable (runs on small datasets)	Can be highly expensive due to massive scale
Core Goal	Evaluates correctness (Is the application built right?)	Evaluates normalcy (Is the application running normally?)

Anatomy of an Online Evaluation Pipeline

Implementing a robust online evaluation pipeline involves the following architectural stages:



Step 1: Logging (The Foundation)

To evaluate a live system, you must first record it. We must capture a structured, replayable record of every single conversation turn. For example, if our website chatbot runs 50,000 conversations daily, we log them in detail.

What We Log:

- **Metadata & IDs:** Conversation ID, Turn ID (tracking individual prompt-response pairs), User ID, Session ID, and Time Stamp.
- **Content Data:** The user's input question, the **retrieved context** (in our RAG pipeline), and the chatbot's generated output.
- **Operational Metrics:** Latency (in milliseconds), prompt tokens used, completion tokens used, total cost, and error status codes.

- **Behavioral & Feedback Metrics:** Thumbs-up/thumbs-down ratings, escalation signals (e.g., users asking to speak to a human or requesting an email address), and question-rephrasing events (users repeatedly entering variations of the same query).

Tooling:

We use **LangSmith** as our production tracing and logging platform to store and view these conversation logs and metadata.

Core Engineering Properties of Logging:

1. **Non-blocking:** Logging operations must run concurrently/asynchronously so that the logging process never blocks or increases response latency for the active user.
 2. **Durable & Queryable:** Logs must be saved in a structured, searchable format in an observability tool or data warehouse so we can fetch them easily in the future.
 3. **Late Signal Attachment:** Many user signals (such as sending a complaint email to support a day after a bad chat) arrive long after the session is closed. We must be able to retroactively link these late-arriving signals back to the original Conversation ID.
 4. **PII Handling (Privacy & Masking):** If a user inputs **Personally Identifiable Information (PII)**—such as phone numbers, credit card numbers, addresses, or ID numbers—this sensitive data must be **masked** or blurred before being stored in LangSmith.
-

Step 2: Tracking Signals (Captured vs. Computed)

Logged data is split into two classes of signals:

1. **Captured Signals:** Metrics that are immediately available in the log environment without any computation. Examples include response latency, token usage counts, API cost, and user feedback (thumbs up/down).
 2. **Computed Signals:** Quality metrics that cannot be directly extracted from logs and must be calculated using an evaluation model. Examples include **faithfulness** (verifying if the answer is strictly grounded in the retrieved context), **answer relevance**, **correctness**, **hallucination**, **toxicity**, and **bias & fairness**.
-

Step 3: Managing Captured Signals (The Fast-Track Pipeline)

Because captured signals require no active calculation, they flow directly through this standard operational monitoring path: $\text{Logging} \rightarrow \text{Dashboarding} \rightarrow \text{Alerting}$

- **Dashboarding:**

- Captured signals are aggregated over time windows (e.g., last hour, 3 hours, 24 hours, or weeks) and displayed on a dashboard. Monitoring individual sessions in isolation is meaningless due to transient network spikes; we focus on aggregated averages and percentiles.
 - If a course launch causes a massive traffic spike and response times climb, the dashboard graph immediately displays the latency spike.
 - **Alerting:**
 - Since developers cannot monitor dashboards 24/7, we set up automated **Alerting Systems** linked to Slack, Email, or PagerDuty.
 - If an aggregated metric crosses a pre-defined threshold over a 5-minute window, the engineering team is instantly alerted to take corrective action—such as spinning up new AWS EC2 instances or adjusting load balancers—to normalize the system.
-

Step 4: Managing Computed Signals (The Evaluation Pipeline)

For computed signals like **hallucination** or **faithfulness**, we run an advanced pipeline: \$\$

$\text{\texttt{Logging}} \rightarrow \text{\texttt{Sampling}} \rightarrow \text{\texttt{Evaluator (LLM as a Judge)}} \rightarrow \text{\texttt{Dashboarding}} \rightarrow \text{\texttt{Alerting}}$ \$\$

1. The Cost Problem & The Necessity of Sampling

- Measuring computed signals online requires running an evaluator. Because online evaluation is reference-free, we must use an **LLM as a Judge**—a powerful LLM guided by a detailed rubric.
- If our live chatbot processes **50,000 conversations a day**, running a powerful LLM to evaluate all 50,000 runs is financially unsustainable and will easily double or triple our API costs.
- To solve this, we implement a **Sampling Step** to select a subset (e.g., 1,000 conversations out of 50,000) to evaluate.
- **Sampling Strategies:**
 - *Random Sampling:* Selecting conversations purely at random. This is simple but highly inefficient.
 - *Stratified Sampling (Our Recommended Strategy):*
 - Instead of random selection, we group our conversations into categories and sample more heavily from **problematic categories** while sampling less from normal, successful interactions.
 - We run a lightweight classifier model or logic to detect signals of interest to group conversations containing:
 - A **thumbs-down** feedback.
 - **Abruptly terminated** conversations.

- Support **escalations**.
- Mentions of high-risk keywords like **pricing, refunds, fees, or admissions**.
- Repeated **rephrasing** of questions.
- (Note: Conversations with explicit thumbs-up ratings can be safely assumed successful and deprioritized in our sample).

2. The Reference-Free Evaluator (LLM as a Judge)

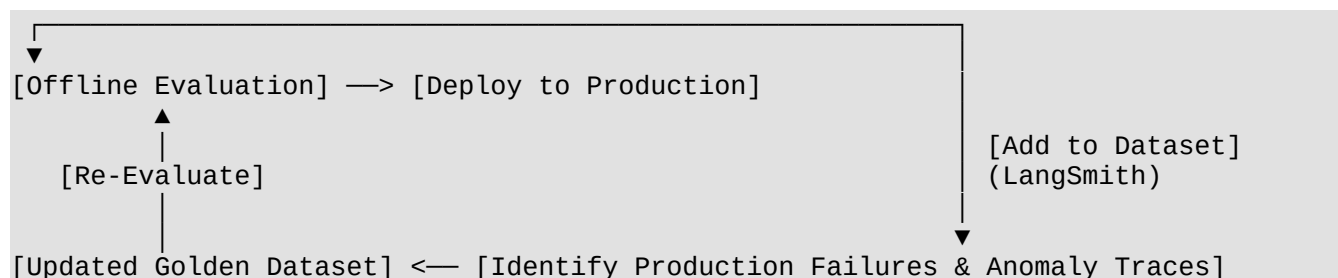
- The sampled conversations are sent to our reference-free evaluator.
- Using a powerful model (e.g., OpenAI or Claude), we provide the user query, retrieved context, generated answer, and a detailed **scoring rubric** to evaluate the metric (such as checking if the response is fully grounded in the retrieved context).
- This is set up in LangSmith by selecting **Tracing** (which hooks it up as a live online evaluator) instead of a **Dataset** (which would run it as an offline evaluator).

3. Dashboarding & Alerting

- The evaluator's outputs (e.g., hallucination rate) are aggregated on our **Dashboard** and tied to our **Alerting System**. If aggregated faithfulness falls below our offline-tested baseline, an alert is triggered so we can troubleshoot.

Closing the Loop: The Self-Improving Evaluation Loop

Online and offline evaluations do not exist in isolation. They form a continuous, **self-improving system loop**:



1. **Offline Evaluation:** We thoroughly test our application against our current golden dataset and deploy it once it passes our release gate.
2. **Production Monitoring & Online Evals:** The application processes live traffic, and online evals capture performance traces.
3. **Identify Failures:** We identify production failures and anomalies (e.g., traces flagged with low faithfulness or thumbs-down feedback) in LangSmith.

4. **Ingest to Golden Dataset:** We use the "**Add to Dataset**" feature in LangSmith to inject these annotated production failures directly back into our offline **golden dataset**.
5. **Continuous Hardening:** The next offline evaluation tests our software against this updated, harder dataset, closing the loop and preventing future production regressions.